aidd-tutorial-creator — Manual completo

Tutoriais animados de qualquer plataforma, a partir de um pedido em linguagem natural

Skill agnóstica · REA · HyperFrames · voz neural · legendas queimadas

Sumário

1. O que é

- 1.1 A ideia central
- 1.2 De onde vem a metodologia

2. Instalação

- 2.1 Onde a skill mora
- 2.2 Dependências
- 2.3 Chave gratuita do Gemini (opcional, recomendada)

3. Primeiro tutorial em 5 minutos

4. Como pedir

- 4.1 Linguagem natural
- 4.2 Com flags

5. Flags (referência completa)

6. O pipeline, fase a fase

- 6.1 Por que essa ordem

7. Captura por tipo de alvo

- 7.1 Regras de segurança e privacidade
- 7.2 O detalhe que mais estraga tutoriais

8. Voz e narração

- 8.1 Motores
- 8.2 Catálogo de vozes: ouvir e escolher
- 8.3 Regras do roteiro

9. Storyboard e layouts

10. Prompt mestre

11. Estilos visuais

- 11.1 Design system do produto (trava de marca)

12. Legendas

13. Verificação

14. Revisão e memória do projeto

15. Modo catálogo: todos os tutoriais de um produto

16. Saídas: formatos, resoluções e canais

17. Referência do `tutorial.py`

- 17.1 Estrutura de um projeto
- 17.2 Fluxo manual completo (sem agente)

18. Receitas para o potencial máximo

19. Solução de problemas

20. Limitações conhecidas

21. Glossário

1. O que é

A `aidd-tutorial-creator` é uma skill que faz um agente de IA produzir um **tutorial animado completo** (MP4 com narração, legendas queimadas e `.srt`) a partir de um pedido como:

> cria um tutorial de 45 segundos mostrando como gerar um relatório em http://localhost:5173/relatorios

O agente não inventa telas: ele **captura a plataforma real** (rota web, site, app Electron, app desktop, app Android, vídeo), escreve o roteiro com os rótulos exatos da interface, gera a voz, anima as telas capturadas em código e verifica o resultado antes de entregar.

1.1 A ideia central

Vídeo é um site animado gravado. O agente escreve HTML, CSS e JavaScript (GSAP para movimento, Three.js e shaders para 3D); um navegador headless desenha cada quadro em função de um tempo `t` e o ffmpeg monta o MP4. Consequências práticas:

- **Custo de iteração baixo.** Mudar uma cena custa alguns tokens e um novo render local, não uma nova geração paga num modelo de vídeo.
- **Determinismo.** Mesmo projeto, mesmo vídeo. Nada depende do relógio ou de aleatoriedade.
- **Fidelidade.** As telas do tutorial são as telas reais do produto, com os textos reais.

1.2 De onde vem a metodologia

A skill condensa 9 vídeos de referência, transcritos localmente:

Fonte	O que trouxe
Matt Penny, *Claude + Three.js = Premium 3D Motion Graphics*	vídeo como site gravado; prompt em 8 blocos; checar stills antes de renderizar
kaye.creatives, *Claude Motion / Opus 5.5*	o prompt é 90 % do trabalho; referências + marca + direção; iteração travada
Video Editors… adopt Claude Code (HyperFrames)	site → vídeo; o agente escreve o próprio prompt; passos pequenos
Claude Code Just Automated My Video Production	screenshots reais; referência → tokens de design; componentes compartilhados
@theguneytopcu (5 vídeos)	voz primeiro; quadros-chave A→D; legendas locais; auditoria roteiro × transcrição; cena piloto; "toda falha vira regra"

2. Instalação

2.1 Onde a skill mora

A pasta `aidd-tutorial-creator/` é a fonte. Os scripts `sync.ps1` (Windows) e `sync.sh` (macOS/Linux) copiam a pasta para todas as harnesses instaladas e registram o comando `/aidd-tutorial-creator`:

```
powershell -File sync.ps1 -DryRun   # mostra o que faria
powershell -File sync.ps1           # instala
sh sync.sh --dry-run
sh sync.sh
```

Destinos: `~/.agents/skills` (sempre), `~/.claude/skills`, `~/.config/opencode/skills`, `~/.gemini/skills`, `~/.gemini/antigravity/skills`, `~/.hermes/skills`, `~/.cline/skills`, `~/.factory/skills`, `~/.mimocode/skills` (quando a harness existe) e os comandos em `~/.claude/commands`, `~/.gemini/commands` e `~/.gemini/antigravity/commands`.

Edite sempre a fonte e rode o `sync` de novo para propagar.

2.2 Dependências

Verifique tudo, sem alterar nada:

```
python scripts/tutorial.py doctor
```

Componente	Para quê	Obrigatório?
Python 3.10+	scripts da skill (só biblioteca padrão)	sim
ffmpeg com libass	frames, áudio, legendas queimadas	sim
Node.js + `npx hyperframes`	composição, checagem, render	sim
REA (servidor MCP `rea` ou `npx rea-agents`)	captura do alvo	sim
Chrome ou Edge	captura web e render	sim
`edge-tts`	voz neural grátis	recomendado
`faster-whisper`	tempos por palavra dos motores sem marcação; transcrição	recomendado
`yt-dlp`	vídeos do YouTube como fonte ou referência	para alvos de vídeo
`google-genai` + `GEMINI_API_KEY`	voz Gemini	opcional
`FISH_API_KEY`	voz Fish Audio	opcional
`kokoro-onnx` + `soundfile`	voz local Kokoro	opcional

Componente	Para quê	Obrigatório?
`adb`	apps Android	para Android
Ghidra (`GHIDRA_INSTALL_DIR`)	análise estática de binário nativo	raramente

A skill **nunca instala nada sem a sua confirmação**.

2.3 Chave gratuita do Gemini (opcional, recomendada)

1. Acesse aistudio.google.com e entre com sua conta Google.
2. Clique em **Get API key** e crie uma chave.
3. Defina a variável de ambiente `GEMINI_API_KEY` (Windows: Configurações > Sistema > Sobre > Configurações avançadas > Variáveis de ambiente; macOS/Linux: `export GEMINI_API_KEY=...` no perfil do shell).
4. Reinicie a harness e rode `tutorial.py doctor`: a linha “chave Gemini” deve aparecer como OK.

3. Primeiro tutorial em 5 minutos

Na harness (Claude Code, Codex, Gemini…):

```
/aidd-tutorial-creator tutorial de 30 s mostrando como filtrar relatórios em
http://localhost:5173/relatorios
```

O que acontece:

1. **Entrevista curta (P1).** O agente pergunta só o que falta (objetivo, passos, duração, voz, estilo) e mostra o brief. Responda “ok” ou corrija.
2. **Captura (P2).** Ele abre o app, executa o fluxo e salva um screenshot antes e depois de cada passo. Você aprova a lista de passos.
3. **Roteiro e voz (P3).** Ele escreve a narração com os rótulos exatos da tela e gera a voz. Você ouve e aprova.
4. **Storyboard (P4).** Tabela de cenas com tempo, layout e o que acontece em cada uma. Você aprova.
5. **Cena piloto (P5).** Uma cena pronta, com o visual final, para você julgar cor, ritmo e tamanho de texto antes do lote.
6. **Build, render, legendas e auditoria.** Sem intervenção.
7. **Entrega e revisão (P6).** Você assiste e pede ajustes em linguagem natural; ele muda só o que você pediu.

Para pular aprovações intermediárias: `--gates essenciais` (só P2, P4 e P6) ou `--gates nenhum` (piloto automático; as verificações automáticas continuam rodando).

4. Como pedir

4.1 Linguagem natural

Diga o **alvo**, o **objetivo** e, se quiser, a **duração** e o **público**. Exemplos:

Pedido	O que a skill entende
“tutorial de 45 s mostrando como gerar um relatório em http://localhost:5173/relatorios”	app web, rota W1
“explainer de 30 s da landing https://meusite.com para Reels”	site, rota W2, 9:16
“tutorial do Notion desktop criando uma página”	Electron, rota E1
“tutorial do meu app Windows Faturador.exe emitindo uma nota”	desktop nativo, rota N1
“tutorial vertical de 30 s do app Android com.exemplo.app ensinando o cadastro”	Android, rota A1
“transforma este vídeo do YouTube num tutorial animado de 1 minuto: [link do vídeo]”	vídeo como conteúdo, rota V1
“quero o tutorial no estilo deste vídeo: [link do vídeo]”	vídeo como referência de estilo, rota V2
“só o roteiro e o storyboard”	para em P4
“ajusta a cena 3, está rápida”	iteração travada direto (lê `ESTADO.md`)
“faz a versão vertical do mesmo tutorial”	reaproveita captura e blueprint, refaz voz, cenas e render no novo formato

4.2 Com flags

Flags vão no próprio pedido e dispensam as perguntas correspondentes:

```
/aidd-tutorial-creator http://localhost:5173/relatorios --duracao 40 --voz gemini:Charon --
legenda bold-pop --formato 9:16 --estilo glass-3d
```

5. Flags (referência completa)

Flag	Valores	Padrão	Efeito
`--voz`	`edge[:voz]`, `gemini[:voz]`, `fish[:reference_id]`, `kokoro[:voz]`, `sistema`, `nenhuma`, `gravacao`	`auto`	motor de narração (seção 8)
`--velocidade`	número (ex.: `1.1`)	`1.0`	ritmo da fala
`--direcao`	texto	linha `Direção:` do roteiro	como ler (o Gemini segue a direção)
`--legenda`	`highlight`, `bold-pop`, `word-by-word`, `clean`, `boxed`, `nenhuma`	`highlight` (16:9) / `bold-pop` (9:16)	estilo da legenda queimada
`--caixa`	`normal`, `alta`, `baixa`	`normal`	caixa do texto da legenda
`--legenda-posicao`	`base`, `meio`, `topo`	`base`	onde a legenda fica
`--legenda-tamanho`	`p`, `m`, `g`	`m`	tamanho da legenda
`--acento`	cor hex	`#FFD43B`	cor da palavra ativa na legenda
`--formato`	`16:9`, `9:16`, `1:1`, `4:5`	`16:9`	proporção e resolução
`--fps`	`24`, `25`, `30`, `50`, `60`	`30`	quadros por segundo
`--duracao`	segundos	do brief	alvo de duração (±10 %)
`--estilo`	`clean-product`, `glass-3d`, `clay-soft`, `paper-editorial`, `dark-tech`	`clean-product`	linguagem visual (seção 11)
`--idioma`	`pt-BR`, `en-US`, `es-ES` …	`pt-BR`	idioma do roteiro, voz e legendas
`--musica`	caminho de arquivo ou `nenhuma`	`nenhuma`	trilha de fundo (sua)
`--sfx`	`on`, `off`	`on`	cliques, pops e whooshes gerados localmente
`--qualidade`	`rascunho`, `final`	`final`	velocidade × qualidade do render
`--gates`	`todos`, `essenciais`, `nenhum`	`todos`	quantas aprovações você quer dar

Flag	Valores	Padrão	Efeito
`--canal`	`instagram-reels`, `instagram-feed`, `tiktok`, `youtube-shorts`, `youtube`, `plataforma-ensino`, `central-de-ajuda`, `linkedin`, `whatsapp`, `apresentacao`	—	onde o vídeo será usado: define formatos, resolução, legenda, zonas seguras e volume (seção 16)
`--formatos`	`16:9`, `9:16`, `1:1`, `4:5` (vários, com vírgula)	o `--formato`	saídas por proporção
`--resolucoes`	`480p`, `720p`, `1080p` (vários)	`1080p`	saídas por resolução
`--responsivo`	`auto`, `sim`, `nao`	`auto`	usa a tela real de celular/tablet nos formatos verticais e quadrados quando o app é responsivo (seção 16)

As flags ficam gravadas em `tutorials/<slug>/tutorial.config.json`. Para mudar depois:

```
python scripts/tutorial.py config tutorials/<slug> --legenda boxed --legenda-posicao topo
```

6. O pipeline, fase a fase

Fase	O que o agente faz	O que você decide
T0 Preparar	`doctor`, cria `tutorials/<slug>/` com as flags, lê `rules.md`	nada
T1 Entrevista	até 5 perguntas, só as que faltam; escreve `brief.md`	**P1**: aprova o brief
T2 Captura	executa o fluxo no alvo; screenshot antes e depois de cada passo; textos reais da UI	**P2**: confirma passos e ordem
T3 Estilo	tokens do produto (cores, fontes, raio) + referência opcional; `blueprint.md`	opcional
T4 Roteiro e voz	narração com rótulos exatos; TTS; mede as durações reais	**P3**: aprova texto e voz
T5 Storyboard	cenas cronometradas, layout por cena, quadros-chave; auditoria do storyboard	**P4**: aprova as cenas
T6 Piloto	componentes compartilhados, prompt mestre e 1 cena completa	**P5**: aprova o visual
T7 Build	demais cenas seguindo o plano travado	nada
T8 Acabamento	render, legendas queimadas, `.srt`, auditoria final	nada
T9 Revisão	ajustes pedidos por você, só no que mudou; lições em `rules.md`	**P6**: aceite

Tudo fica registrado em `ESTADO.md`. Se a conversa cair, peça "continua o tutorial X": o agente lê o estado e retoma da fase em que parou.

6.1 Por que essa ordem

- **Voz antes das cenas.** A fala define quanto tempo cada cena dura, e o clique do cursor acontece no instante em que a narração diz o nome do botão.
- **Stills antes de movimento, piloto antes do lote.** Corrigir um quadro parado é barato; corrigir um render inteiro não.
- **Componentes compartilhados antes das cenas.** Janela, cursor, callout e chip de passo são os mesmos em todas as cenas: o vídeo parece uma peça só.

7. Captura por tipo de alvo

Rota	Alvo	Como	Sua participação
W1	app web	REA `capture_browser_scenario`: uma ação por passo, screenshot e árvore de acessibilidade a cada passo	áreas com login: você abre o Chrome com `--remote-debugging-port=9222` e faz login; o agente se conecta sem tocar na senha
W2	site público	`npx hyperframes capture <url>` (telas, cores, fontes, componentes) + REA	nenhuma
E1	Electron	app aberto com `--remote-debugging-port=9223`, captura pelo mesmo contrato web; estrutura pelo `.asar`	abrir o app com a flag
N1	desktop nativo	macOS: REA observa a UI pela acessibilidade. Windows/Linux: `tutorial.py record` grava a janela enquanto você executa o fluxo; `tutorial.py frames` extrai um quadro por mudança de tela	executar o fluxo durante a gravação
N2	binário nativo (raro)	REA + Ghidra: strings e telas que não aparecem observando a UI	autorizar
A1	Android	REA via ADB: abre o app, captura a tela a cada passo	aparelho ou emulador conectado; você executa os passos
I1	iOS	só análise estática do `.ipa` + gravações de tela suas	gravar a tela do iPhone
V1	vídeo como conteúdo	`tutorial.py reference`: baixa, transcreve e extrai quadros; o agente monta os passos	direitos sobre o vídeo
V2	vídeo como referência de estilo	mesmo comando; o agente extrai só paleta, ritmo, câmera, transições e tipografia	nenhuma

7.1 Regras de segurança e privacidade

- **Login é seu.** O agente nunca digita senha. Segredos, se necessários, só por variável de ambiente.
- **Ações que gravam dados** (criar, enviar, apagar, pagar) exigem sua autorização por ação; prefira uma conta de teste.
- **Dados pessoais** que aparecem na tela são substituídos por dados fictícios.

- **Vídeos de terceiros** servem como roteiro ou referência; o tutorial recria os visuais, nunca recorta a filmagem de outra pessoa.

7.2 O detalhe que mais estraga tutoriais

O REA captura a tela **depois** de cada ação. Durante o clique, o vídeo deve mostrar a tela **de antes**; a tela de depois aparece no instante do clique. A skill registra as duas (`evidence_before` e `evidence`) e a composição troca de uma para a outra no clique.

8. Voz e narração

8.1 Motores

Motor	Qualidade	Custo	Observações
`edge`	neural, natural	grátis, sem chave	vozes pt-BR: `pt-BR-FranciscaNeural`, `pt-BR-AntonioNeural`, `pt-BR-ThalitaMultilingualNeural`. Devolve o tempo exato de cada palavra. Envia o texto da narração ao serviço online da Microsoft
`gemini`	muito expressiva, segue a direção de leitura	grátis com chave do Google AI Studio	vozes: `Kore`, `Charon`, `Puck`, `Aoede`, `Fenrir` … Envia o texto ao Google
`fish`	alta, permite clonar voz	plano grátis limitado	use o `reference_id` de um modelo de voz do fish.audio
`kokoro`	boa, 100 % local	grátis	requer `pip install kokoro-onnx soundfile`
`sistema`	robótica	grátis	último recurso
`gravacao`	sua voz	—	o agente entrega o roteiro como teleprompter; a transcrição alinha as cenas
`nenhuma`	—	—	só texto na tela e música

`auto` tenta edge → gemini → kokoro → sistema e avisa a cada queda. Motor escolhido explicitamente não cai para outro: se falhar, a skill explica a causa.

8.2 Catálogo de vozes: ouvir e escolher

Peça "quero ouvir as vozes" ou rode:

```
python scripts/tutorial.py voices <pasta> --motores edge,gemini,system --amostras --texto "Olá!
        Eu sou a voz dos tutoriais do meu produto."
```

O `VOZES.md` lista cada voz com gênero, caráter, idioma, um link para a **amostra em áudio** e a flag pronta para usar. No laboratório: 47 vozes e 17 amostras em cerca de 30 segundos.

- **Nativas pt-BR (edge):** Antonio (masculina), Francisca (feminina) e Thalita (feminina).
- **Multilíngues (edge):** Andrew, Ava, Emma, Brian e outras, que falam português com leve sotaque. Podem dar um tom diferente.
- **Gemini:** 30 vozes com caráter (calorosa, firme, informativa, animada…). As amostras exigem a chave gratuita (seção 2.3).

Depois de escolher:

```
python scripts/tutorial.py config tutorials/<slug> --voz edge:pt-BR-AntonioNeural
```

A escolha fica no `tutorial.config.json` e no Registro de decisões do `ESTADO.md`. No modo catálogo, a voz do produto vale para todos os tutoriais.

Para o Gemini, a direção de leitura faz diferença: `--direcao "caloroso, confiante, ritmo médio, sorriso na voz"`.

8.3 Regras do roteiro

- **2,5 palavras por segundo** (25 a cada 10 s). `tutorial.py timing` mostra se o roteiro cabe na duração.
- **Verbo de ação + alvo visível + resultado:** "Clique em Período e escolha Últimos 30 dias. A tabela se atualiza na hora."
- **Rótulos exatos da interface**, como aparecem na tela.
- Frases de até 14 palavras, segunda pessoa, presente, voz ativa.
- Estrutura: gancho (2–4 s), contexto (opcional), passos, resultado, fecho.

Formato do `script.md`:

```
# Roteiro — Seu primeiro relatório
Direção: caloroso, confiante, ritmo médio.

## c01 — Gancho
Em poucos segundos, você abre | seu primeiro relatório.

## c02 — Abrir relatórios  [steps: s01]
Clique em Relatórios, no menu da esquerda.
```

O `|` força uma quebra de legenda naquele ponto; a voz não lê o marcador.

9. Storyboard e layouts

Cada cena tem um layout do menu abaixo; o agente evita repetir o mesmo layout mais de duas vezes seguidas.

Layout	Para quê
`title`	abertura, gancho
`app-full`	contexto: onde estamos
`zoom-focus`	a ação principal: zoom no elemento, cursor clica
`callout`	explicar um elemento com uma etiqueta ancorada
`cursor-path`	2 ou 3 cliques próximos em sequência
`split`	antes/depois, lista de opções
`step-list`	visão geral dos passos
`device-frame`	tela de celular dentro de um aparelho, em 16:9
`pip`	apresentador num canto, tela do app no resto
`result`	estado final com destaque
`outro`	fecho, marca, chamada para ação

O `storyboard.json` é verificado por máquina (`tutorial.py audit storyboard`): cenas contíguas, cada passo em exatamente uma cena, narração que cabe no tempo, textos curtos na tela, layouts válidos.

10. Prompt mestre

Antes do build, o agente gera `composition/PROMPT.md` em 8 blocos, sempre nesta ordem: cabeçalho técnico (duração, resolução, fps, tempo determinístico), mundo e estilo, camada de texto, câmera, cenas cronometradas, elementos contínuos, acabamento e verificação. As regras do `rules.md` entram no fim e vencem qualquer outra instrução.

Se você não tem preferência visual, diga “surpreenda-me”: o agente propõe 3 direções em texto antes de construir.

11. Estilos visuais

Estilo	Quando usar	Visual
`clean-product`	SaaS, painéis, documentação	fundo claro com gradiente sutil da cor da marca; janela do app flutuando; cursor, zoom e callouts
`glass-3d`	lançamentos, features premium	espaço escuro, painéis de vidro fosco em 3D, câmera que voa entre telas
`clay-soft`	onboarding, produtos amigáveis	estúdio claro, materiais pastel, elementos que se materializam
`paper-editorial`	explicações conceituais	papel com grade de pontos, recortes, marca-texto
`dark-tech`	ferramentas de desenvolvedor, APIs	fundo escuro, fonte mono, brilho ciano

Para usar o estilo de um vídeo existente: "no estilo deste vídeo: [link do vídeo]". O agente extrai só paleta, ritmo, movimentos e tipografia; nunca copia história nem marca.

11.1 Design system do produto (trava de marca)

O tutorial nunca usa o visual de outra empresa. A skill extrai o design system **real** do produto e trava a composição nele:

```
npx hyperframes capture https://seu-produto -o tutorials/<slug>/capture --skip-vision --max-
        screenshots 4
python scripts/tutorial.py design-tokens tutorials/<slug>/capture --out tutorials/<slug>
python scripts/tutorial.py audit brand tutorials/<slug>
```

- `design-tokens.json`: cores nomeadas pelas variáveis CSS do produto (primária, acento, texto, fundo, sucesso, erro...), paleta completa, fontes, raios, sombras e logo, com a origem de cada valor.
- `brand/brand.css`: as fontes do produto (arquivos locais, com as mesmas regras do site) e as variáveis `--brand-*`. Tudo que a skill desenha por cima das telas (cursor, callouts, chips, títulos, fecho, cor da legenda) usa só essas variáveis.
- `audit brand`: reprova qualquer cor ou fonte da composição que não esteja no design system. No teste de validação, pegou as três sabotagens injetadas (duas cores e uma fonte de outra marca).
- Em áreas com login, capture uma tela pública do mesmo produto (login ou landing); o design system costuma ser o mesmo.
- No modo catálogo, a extração acontece uma vez e fica em `compartilhado/`, valendo para todos os tutoriais do produto.

12. Legendas

As legendas são queimadas depois do render, a partir do tempo exato de cada palavra da narração.

Estilo	Visual
`highlight`	frase curta; a palavra falada ganha uma pílula na cor de acento
`bold-pop`	3 palavras grandes; a palavra falada acende e cresce
`word-by-word`	as palavras aparecem conforme são ditas
`clean`	legenda clássica, estática
`boxed`	texto claro numa faixa escura translúcida

Controles: `--caixa`, `--legenda-posicao`, `--legenda-tamanho`, `--acento` e o marcador `|` no roteiro.

- **Posição:** `topo` quando há algo importante na parte de baixo da tela (barra de ações, rodapé, teclado no celular); `meio` só em cenas sem conteúdo central, porque pode cobrir título ou formulário.
- **Tamanho:** `g` para vídeos verticais vistos no celular.
- **Acento:** o amarelo padrão contrasta com qualquer fundo; a cor da marca funciona melhor no `highlight` do que no `bold-pop`.

Trocar de estilo não exige novo render:

```
python scripts/tutorial.py burn tutorials/<slug> --style bold-pop --position top --size g --out
        renders/variantes/topo.mp4
```

O `.srt` sai sempre junto, para o YouTube (tradução automática e acessibilidade).

13. Verificação

Quando	Verificação	Reprova se
T5	`tutorial.py audit storyboard`	buraco ou sobreposição na linha do tempo; passo ausente ou repetido; narração longa demais para a cena; texto de tela com mais de 7 palavras; layout inválido; evidência faltando
T6, T7	`tutorial.py audit brand`	cor ou fonte fora do design system do produto
T6, T7	`npx hyperframes check`	erros de lint, runtime, layout (texto cortado, sobreposto, encoberto), contraste; áudio sem `id` (sairia mudo)
T6, T7	inspeção dos stills pelo agente	texto fora do quadro, pequeno, sobreposto; cursor no lugar errado; dado sensível; estado de tela errado
T8	`tutorial.py audit final`	duração, resolução ou fps diferentes do plano; sem áudio; sem `.srt`; legenda que atravessa troca de cena; legenda queimada faltando quando a flag pede
voz gravada	`tutorial.py audit coverage`	frase do roteiro ausente ou repetida (retake) no áudio

A entrega termina com um veredito: **VERIFIED**, **VERIFIED WITH CAVEATS** (com o que não foi verificado) ou **REFUTED**.

14. Revisão e memória do projeto

Peça ajustes como pediria a um editor: "a cena 3 está rápida", "não gostei desta transição", "a legenda está cobrindo o botão". O agente:

1. diz em uma linha o que vai mudar e o que fica intocado;
2. altera só aquilo;
3. roda as verificações nas cenas tocadas e renderiza de novo.

O arquivo `rules.md` do projeto guarda três listas, lidas em toda execução:

- **Congelado:** cenas aprovadas que não mudam sem pedido explícito.
- **Lições:** cada erro corrigido vira uma regra curta ("callout nunca cobre o botão clicado").
- **Preferências:** seu gosto ("prefiro cortes secos").

Para um novo tutorial do mesmo produto, o agente reaproveita `blueprint.md`, componentes e `rules.md`: o segundo vídeo sai mais rápido e sem os erros do primeiro.

15. Modo catálogo: todos os tutoriais de um produto

Peça “quero os tutoriais da minha aplicação https://...” e a skill:

1. **Reconhece** a página (há login? há atalhos de teste? quais perfis?).
2. **Pede autorização para entrar**: atalhos de teste da própria tela, a sua sessão no Chrome (`--remote-debugging-port=9222`) ou só páginas públicas. Nunca digita senha.
3. **Mapeia cada perfil só observando** (`scripts/explore.py`): clica apenas em visualizações, filtros, abas, ordenação e paginação, e abre links internos por endereço. Botões de ação (salvar, enviar, gerar, excluir, inativar, exportar, sair) são **listados sem clicar**.
4. **Extrai o design system** uma vez para o produto inteiro.
5. **Propõe o menu** em `CATALOGO.md` : tutoriais por jornada e perfil, com duração, tipo (observação ou ação), status, saídas já geradas, o comando de cada um e a ordem do “gerar todos”. Também registra os achados (telas quebradas, dados pessoais à vista).

```
<produto>/                        ex.: cadastros-conexao/
├── CATALOGO.md · catalogo.json
├── mapa-do-app/                  capturas da análise por perfil
├── compartilhado/                design system único
└── <tutorial-em-kebab-case>/     um projeto por tutorial
```

Você pede	A skill faz
`gerar <pasta>`	produz esse tutorial com os padrões do produto
`gerar todos`	produz um de cada vez, na ordem do catálogo; para nos itens que exigem uma ação real e pede autorização; retoma de onde parou
`regenerar <pasta>`	refaz o tutorial
`adicionar "<fluxo>"`	captura o fluxo e inclui no catálogo
`remover <pasta>`	tira do catálogo (a pasta fica)
`refazer catalogo`	analisa de novo e marca como desatualizados os tutoriais cujas telas mudaram

No laboratório `cadastros.vpsconexao.org` , o mapeamento de 5 perfis gerou um catálogo de 16 tutoriais em 5 jornadas e apontou 2 defeitos do próprio app (um atalho de teste inoperante e o formulário demo sem conexão).

16. Saídas: formatos, resoluções e canais

Cada tutorial tem um vídeo-mestre e quantas saídas você quiser. A forma mais simples é dizer **onde o vídeo será usado**:

```
/aidd-tutorial-creator gerar acessar-o-sistema --canal instagram-reels,plataforma-ensino
```

Canal	Saídas	Legenda	Observações
Instagram Reels	9:16 1080p	bold-pop grande	legenda acima da interface do app (22 % da base) · até 90 s
Instagram feed	4:5 1080p	highlight	até 60 s
TikTok	9:16 1080p	bold-pop grande	zona segura maior na base (25 %)
YouTube Shorts	9:16 1080p	bold-pop grande	até 180 s
YouTube	16:9 1080p	highlight	—
Plataforma de ensino / EAD	16:9 1080p e 720p	boxed	volume mais baixo (-16 LUFS), `.srt`
Central de ajuda / docs	16:9 1080p e 720p	clean	—
LinkedIn	1:1 1080p	highlight	—
WhatsApp	9:16 720p	bold-pop grande	arquivo leve (até 2 Mbit/s)
Apresentação	16:9 1080p	sem legenda queimada	`.srt` separado

Ou escolha diretamente: `--formatos 16:9,9:16,1:1 --resolucoes 720p,1080p`.

Adequar um tutorial já pronto (sem refazer o trabalho):

```
python scripts/tutorial.py export <produto>/<tutorial> --formatos 9:16,1:1        # de 16:9 para vertical e quadrado
python scripts/tutorial.py export <produto>/<tutorial> --resolucoes 720p,480p     # de 1080p para resoluções menores
python scripts/tutorial.py export <produto>/<tutorial> --canal whatsapp
python scripts/tutorial.py export <produto>/<tutorial> --legenda boxed            # trocar o tipo de legenda
```

Três modos de adequação:

- **escalar:** mesma proporção, outra resolução. Redimensiona com filtro de alta qualidade e nitidez leve.
- **reenquadrar:** outra proporção, em segundos. Recorta a zona central do vídeo e encaixa no novo quadro sobre o fundo desfocado. Fica ótimo em 1:1 e 4:5; em 9:16 a interface fica menor.
- **nativo:** o agente recompõe as cenas para o novo formato, com zooms e layout próprios. É o melhor para Reels, Shorts e TikTok.

Captura responsiva (`--responsivo auto|sim|nao`). Se o seu app funciona em celular, o vídeo vertical deve mostrar a **tela de celular de verdade**, não um recorte da versão desktop. Com `auto` (padrão), a skill testa o app em cada tamanho pedido: tag de viewport, rolagem lateral, elementos vazando da tela e tamanho do menor texto. Depois decide entre captura nativa e reenquadramento. O veredito por tela é **sim**, **parcial** (por exemplo, uma tabela larga que rola por dentro) ou **não**.

```
python scripts/tutorial.py responsive https://seu-app --out <pasta> --formatos 9:16,1:1 --projeto
        tutorials/<slug>
python scripts/tutorial.py responsive https://seu-app --out <pasta> --formatos 9:16 --atalho
        "Consultor"   # área logada
```

No laboratório: login, recuperação de senha, Painel do Consultor, Fila de Cadastros e Templates deram **sim**; Personalizar Formulários deu **parcial** (a tabela de campos corta colunas). O tutorial de acesso em 9:16 foi refeito com as telas reais de celular. O menor texto da interface passou de cerca de 20 px (reenquadrado) para 28 px (nativo), e nada fica cortado nas bordas.

Legibilidade garantida por medição. Cada exportação calcula o tamanho do menor texto da interface naquela saída e classifica como legível (≥ 14 px), no limite (10–14 px) ou ilegível (< 10 px). No teste, 16:9 em 720p deu 16 px (legível), em 480p 10,7 px (no limite) e as versões 9:16 e 1:1 reenquadradas deram 20 px (legíveis). As legendas são queimadas já no tamanho final de cada saída; quando a cor da marca é escura, a palavra em destaque fica branca com contorno na cor da marca, para não perder contraste.

Tudo fica registrado. As escolhas (canais, formatos, resoluções, legenda) ficam em `tutorial.config.json`, e cada saída gerada (arquivo, dimensões, modo, legenda, canal, legibilidade) entra no "Registro de decisões" do `ESTADO.md`. O `CATALOGO.md` mostra as saídas de cada tutorial.

17. Referência do `tutorial.py`

Todos os comandos: `python scripts/tutorial.py <comando> --help`.

Comando	O que faz
`doctor`	verifica dependências, vozes, chaves e libass (somente leitura)
`init <dir> [flags]`	cria o projeto e grava `tutorial.config.json`
`config <dir> [flags]`	mostra ou altera as flags gravadas
`timing <script.md> [--wps --target]`	estima a duração da narração por cena
`tts <script.md> [--engine --voice --speed --lang --model]`	gera a voz por cena, `words.json`, `durations.json`
`retime <projeto> [--gap]`	recalcula os tempos das cenas pelas durações reais
`narration <projeto>`	monta `audio/narration.wav` na linha do tempo
`frames <vídeo> --out [--scene --every --at --sheet]`	extrai quadros (por mudança de cena, intervalo ou tempos)
`record --seconds --out [--window --device --fps]`	grava a tela ou uma janela
`reference <url-ou-arquivo> --out [--lang]`	baixa, transcreve e extrai quadros de um vídeo
`captions <mídia> [--lang --model]`	transcreve e gera `.srt`
`extract-rea <resultado.json> <pasta>`	salva os screenshots de uma captura do REA, nomeados por passo
`design-tokens <captura> --out <pasta>`	extrai o design system do produto: `design-tokens.json` + `brand/brand.css`
`sfx <pasta>`	gera `click`, `pop`, `whoosh`, `tick` localmente
`sync-composition <projeto>`	grava `timeline.js` e ajusta os tempos da composição pelo storyboard
`render <projeto> [--quality] [--composicao <pasta> --formato <f>]`	renderiza o mestre `renders/<slug>.raw.mp4` (ou o mestre nativo de outro formato); pasta temporária local, até 3 tentativas
`export <projeto> [--canal --formatos --resolucoes --legenda --modo]`	gera as saídas `<slug>-<formato>-<res>.mp4` com legenda, loudness e checagem de legibilidade; registra no `ESTADO.md`
`catalog render|next|add|remove|set <produto>`	mantém o catálogo do produto e gera o `CATALOGO.md`
`voices <pasta> [--motores --idioma --amostras --texto]`	catálogo de vozes com amostras em áudio (`VOZES.md`)

Comando	O que faz
`responsive <url> --out <pasta> [--formatos --atalho --caminhos --projeto]`	testa a responsividade por formato (sim, parcial, não) e registra no projeto
`explore.py <saída> --base <url> --perfil <p> --entrar atalho|cdp|publico`	mapeia um perfil do produto só observando (script separado, em `scripts/`)
`burn <projeto> [--style --position --size --case --accent --font]`	queima a legenda e gera `.srt` e `.ass`
`audit storyboard|final|coverage|brand <projeto>`	auditorias (brand = trava de marca)

17.1 Estrutura de um projeto

```
tutorials/<slug>/
├── tutorial.config.json   flags
├── ESTADO.md              fase, gates, decisões, pendências
├── rules.md               congelado, lições, preferências
├── brief.md · blueprint.md · script.md
├── step-map.json          passos com evidência antes/depois
├── storyboard.json        cenas cronometradas
├── evidence/              capturas do alvo
├── audio/                 <cena>.wav, <cena>.words.json, durations.json, narration.wav
├── composition/           projeto HyperFrames (index.html, timeline.js, assets/)
├── snapshots/             stills de revisão e resultados do check
└── renders/               <slug>.raw.mp4, <slug>.mp4, .srt, .ass, variantes/
```

17.2 Fluxo manual completo (sem agente)

```
python scripts/tutorial.py init tutorials/relatorios --voz edge:pt-BR-FranciscaNeural --legenda
        highlight
# captura com o REA e: python scripts/tutorial.py extract-rea capture.json
        tutorials/relatorios/evidence
# escreva step-map.json, script.md e storyboard.json
python scripts/tutorial.py tts tutorials/relatorios/script.md
python scripts/tutorial.py retime tutorials/relatorios
python scripts/tutorial.py narration tutorials/relatorios
python scripts/tutorial.py audit storyboard tutorials/relatorios
python scripts/tutorial.py sfx tutorials/relatorios/composition/assets
# escreva a composição (index.html com data-scene) e:
python scripts/tutorial.py sync-composition tutorials/relatorios
python scripts/tutorial.py render tutorials/relatorios
python scripts/tutorial.py burn tutorials/relatorios
python scripts/tutorial.py audit final tutorials/relatorios
```

18. Receitas para o potencial máximo

1. **Dê o objetivo, não só a tela.** "Um cliente novo consegue exportar o relatório sozinho" gera um roteiro melhor que "mostra a tela de relatórios".
2. **Use uma conta de demonstração** com dados bonitos e fictícios: a captura já sai pronta para o vídeo.
3. **Comece com** `--qualidade rascunho` para aprovar ritmo e conteúdo; troque para `final` só no render de entrega.
4. **Prefira** `edge` **ou** `gemini`**.** Para o Gemini, escreva uma direção de leitura com personalidade.
5. **Peça cliques sincronizados com a fala.** É o que separa um tutorial profissional de uma apresentação de slides; a skill faz isso por padrão quando há `words.json`.
6. **Gere variantes de legenda** com `burn --out renders/variantes/ …` e escolha assistindo; leva segundos.
7. **Use** `|` **no roteiro** para não separar nome de produto e verbo na legenda.
8. **Uma versão por formato.** Para Reels/Shorts, peça "versão vertical": a skill refaz voz, layout e tamanhos para 9:16 em vez de cortar o 16:9.
9. **Reaproveite o blueprint.** Uma série de tutoriais do mesmo produto fica consistente e cada vídeo sai mais rápido que o anterior.
10. **Referência de estilo de alto nível.** Aponte um trailer de produto que você admira (rota V2); a skill extrai o ritmo e os movimentos sem copiar a marca.
11. **Para 3D premium**, use `--estilo glass-3d` ou `clay-soft`: câmera que voa entre telas, vidro, bloom e grão, mantendo a UI legível durante a ação.
12. **Deixe o** `rules.md` **crescer.** Cada correção vira regra, e o vídeo seguinte já nasce sem aquele erro.

19. Solução de problemas

Sintoma	Causa	Solução
Voz robótica	caiu no motor `sistema`	use `--voz edge` ou `--voz gemini`; rode `doctor`
`edge-tts falhou: 403`	pacote desatualizado	`pip install -U edge-tts`
`GEMINI_API_KEY ausente`	chave não definida	seção 2.3
Gemini: modelo não encontrado	nome do modelo mudou	`--model <nome>` ou `TC_GEMINI_TTS_MODEL`
Render falha com `ENOENT ... hf-render-*` ou `audioPadTrim`	arquivos temporários apagados durante o render	use `tutorial.py render` (pasta temporária local)
Vídeo sem som	`<audio>` sem `id`	o `hyperframes check` acusa `media_missing_id`; adicione `id`
Legenda junta frases de cenas diferentes	`words.json` antigo ou sem pontuação	rode `tts` de novo e depois `burn`; o `audit final` acusa
Legenda cobre um botão	posição padrão	`--legenda-posicao topo` ou um `|` para encurtar o bloco
Tela errada durante o clique	uso do “depois” no lugar do “antes”	`evidence_before` no `step-map.json` (seção 7.2)
Cenas fora de sincronia depois de mudar a voz	tempos antigos	`retime` → `narration` → `sync-composition` → `render`
Detector de cena não extrai quadros	transições suaves	o `frames` cai sozinho para intervalo fixo; ou use `--every`
Saída 480p “no limite” ou “ilegível”	a interface encolhe demais	use 720p ou mais; para 9:16, prefira o modo `nativo`
Palavra ativa da legenda some	cor de acento escura sobre contorno escuro	a skill já troca para texto branco com contorno da marca; ou use `--acento` mais claro
`gerar todos` parou num item	o item exige uma ação real (`tipo: acao`)	autorize a ação descrita ou pule o item; o lote retoma de onde parou
JSON inválido com BOM	arquivo gravado pelo PowerShell	os scripts já aceitam BOM; se outro programa reclamar, salve como UTF-8 sem BOM

20. Limitações conhecidas

Item	Situação
Rota web completa	testada de ponta a ponta
Voz `edge` e voz do sistema	testadas
Legendas: 5 estilos, posição, tamanho, quebra manual	testadas
Auditorias contra projeto sabotado	testadas (9 sabotagens detectadas)
Vozes `gemini`, `fish`, `kokoro`	implementadas, não testadas na máquina de origem (sem chave/pacote)
Rotas Electron, desktop nativo, Android, iOS e o comando `record`	documentadas, não testadas na máquina de origem
Música de fundo	precisa de um arquivo seu; a skill não gera música
Modo catálogo (análise, menu, gerar/adicionar/remover)	testado no laboratório cadastros.vpsconexao.org (5 perfis, 16 tutoriais)
Saídas: escalar, reenquadrar e nativo (9:16 com telas reais de celular)	testadas no tutorial de acesso
Catálogo de vozes com amostras	testado (edge e voz do sistema); amostras do Gemini dependem da chave
Medidor de responsividade	testado em 6 telas do laboratório; vereditos conferidos visualmente
`gerar todos` em lote	o laço está implementado (`catalog next`), mas só 1 tutorial foi gerado até agora
iOS	sem captura ao vivo
Captura de áreas com login	depende de você abrir o navegador com depuração remota

21. Glossário

Termo	Significado
Gate	ponto em que o agente para e espera sua aprovação
Evidência	captura real do alvo (screenshot, texto da UI, árvore de acessibilidade)
Step map	lista de passos do tutorial com a evidência de cada um
Storyboard	plano de cenas com tempos, layout e narração
Cena piloto	primeira cena pronta, usada para aprovar o visual antes do lote
Blueprint	sistema visual do vídeo (cores, fontes, estilo, componentes)
Prompt mestre	especificação em 8 blocos que guia o build
Iteração travada	mudar só o que foi pedido, mantendo o resto intacto
REA	Reverse Engineer Anything: ferramentas de captura e análise de apps
HyperFrames	framework que renderiza HTML animado em vídeo
`words.json`	tempo de cada palavra da narração; base das legendas e da sincronia